

Dictionaries

HOW WOULD YOU LOOK THIS UP?

- You know a person's name and want their birth year
- With lists, that means two lists kept in the same order
- Find the name's position in one list, then use it in the other
- Add a name to one list and forget the other, and every lookup after it is wrong

```
names = ["Ada", "Grace", "Alan"]
born = [1815, 1906, 1912]

i = names.index("Grace")
print(born[i])
```

1906

	0	1	2
names	"Ada"	"Grace"	"Alan"
born	1815	1906	1912

same position, same person

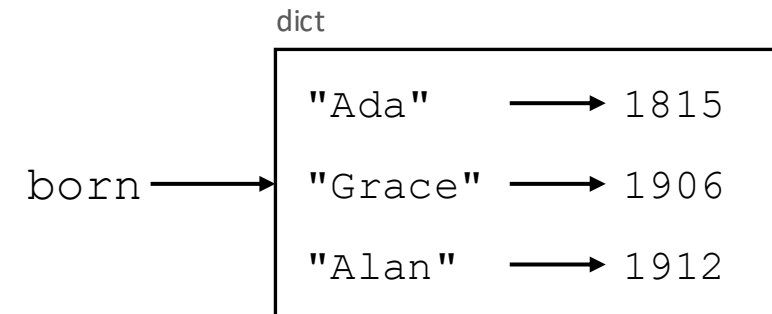
KEYS MAP TO VALUES

- A dictionary is a collection of items; each item is a **key** and the **value** it maps to
- Make one with curly braces: `key: value`, commas between items
- Look a value up by putting its key in square brackets
- A list goes from a position to a value. A dictionary goes from a key to a value

```
born = {"Ada": 1815,  
        "Grace": 1906,  
        "Alan": 1912}  
print(born["Grace"])  
print(len(born))
```

1906

3

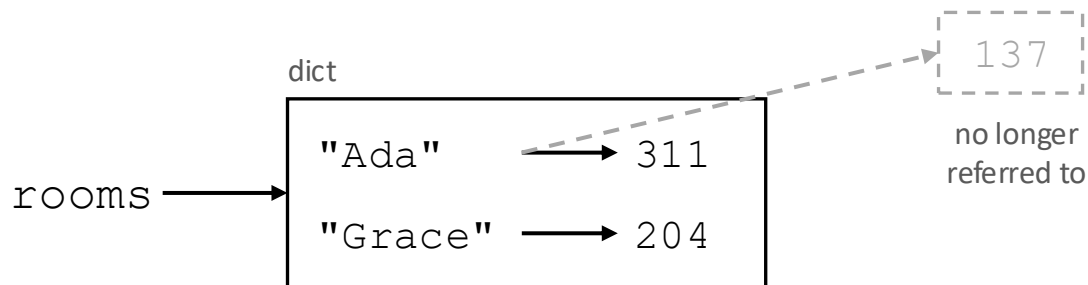


ADDING AND CHANGING ITEMS

- `{ }` is the empty dictionary
- Assigning to a new key adds an item
- Assigning to a key that's already there replaces its value
- Dictionaries are mutable, just like lists

```
rooms = {}  
rooms["Ada"] = 137  
rooms["Grace"] = 204  
rooms["Ada"] = 311  
print(rooms)  
print(len(rooms))
```

```
{'Ada': 311, 'Grace': 204}  
2
```



IN CHECKS KEYS

- `in` asks: is this a key in the dictionary?
- It never looks at the values
- To search the values, use `.values()`

```
born = {"Ada": 1815, "Grace": 1906}
print("Ada" in born)
print(1815 in born)
print(1815 in born.values())
```

True

False

True

Pause and predict: what does line 3 print?

A MISSING KEY IS AN ERROR

- Looking up a key that isn't there raises a `KeyError`
- The error message shows the key it couldn't find
- Keys have to match exactly, including upper and lower case
- Use `in` first if you're not sure the key is there

```
born = {"Ada": 1815, "Grace": 1906}  
print(born["ada"])
```

```
KeyError: 'ada'
```

GET: A FALLBACK VALUE

- `get(key)` looks a key up and returns `None` if it's missing
- `get(key, default)` returns the default instead
- Either way, `get` never adds the key

```
rooms = {"Ada": 137, "Grace": 204}
print(rooms.get("Ada"))
print(rooms.get("Alan"))
print(rooms.get("Alan", 0))
print(rooms)
```

137

None

0

`{'Ada': 137, 'Grace': 204}`

QUESTIONS YOU MIGHT BE ASKING

- Can two keys have the same value? **Yes**
- Can the same key appear twice? **No**: the second one replaces the first
- Can a value be a list? **Yes** (more on that soon)
- Can a key be a list? Hold that thought

```
rooms = {"Ada": 137, "Grace": 137}  
print(rooms)
```

```
oops = {"Ada": 137, "Ada": 204}  
print(oops)
```

```
{'Ada': 137, 'Grace': 137}
```

```
{'Ada': 204}
```

LOOPING OVER A DICTIONARY

- A `for` loop over a dictionary visits its **keys**
- Look each value up inside the loop
- Keys come out in the order they were added

```
rooms = {"Ada": 137, "Grace": 204, "Alan": 311}
for name in rooms:
    print(name, rooms[name])
```

```
Ada 137
Grace 204
Alan 311
```

ADDING UP THE VALUES

- Same accumulator pattern as with lists: start at 0, add as you go
- `sum(d.values())` does the same thing in one line

```
wins = {"X": 3, "O": 1, "tie": 1}
total = 0
for result in wins:
    total += wins[result]
print(total)
print(sum(wins.values()))
```

5

5

BUILDING A DICTIONARY IN A LOOP

- Start with an empty dictionary
- Each pass through the loop adds one item
- Here: turning the two lists from the start into one dictionary

```
names = ["Ada", "Grace", "Alan"]  
years = [1815, 1906, 1912]
```

```
born = {}  
for i in range(len(names)):  
    born[names[i]] = years[i]  
print(born)
```

```
{'Ada': 1815, 'Grace': 1906, 'Alan': 1912}
```

FINDING THE SMALLEST

- Loop over the keys and remember the best key so far
- Compare values, but keep track of the key
- None means "nothing found yet"

```
born = {"Ada": 1815, "Grace": 1906, "Alan": 1912}
oldest = None
for name in born:
    if oldest is None or born[name] < born[oldest]:
        oldest = name
print(oldest)
```

Ada

COUNTING WITH A DICTIONARY

- First time we see a letter: add it with a count of 1
- After that: add 1 to its count
- One dictionary holds a counter for every letter

letter	counter after this letter
"b"	{'b': 1}
"a"	{'b': 1, 'a': 1}
"n"	{'b': 1, 'a': 1, 'n': 1}
"a"	{'b': 1, 'a': 2, 'n': 1}
"n"	{'b': 1, 'a': 2, 'n': 2}
"a"	{'b': 1, 'a': 3, 'n': 2}

```
def value_counts(string):  
    counter = {}  
    for letter in string:  
        if letter not in counter:  
            counter[letter] = 1  
        else:  
            counter[letter] += 1  
    return counter
```

```
print(value_counts("banana"))
```

```
{'b': 1, 'a': 3, 'n': 2}
```

COUNTING WITH GET

- `counter.get(letter, 0)` is the count so far, or 0 if the letter is new
- Add 1 and store it back: one line, no `if`

```
def value_counts(string):  
    counter = {}  
    for letter in string:  
        counter[letter] = counter.get(letter, 0) + 1  
    return counter
```

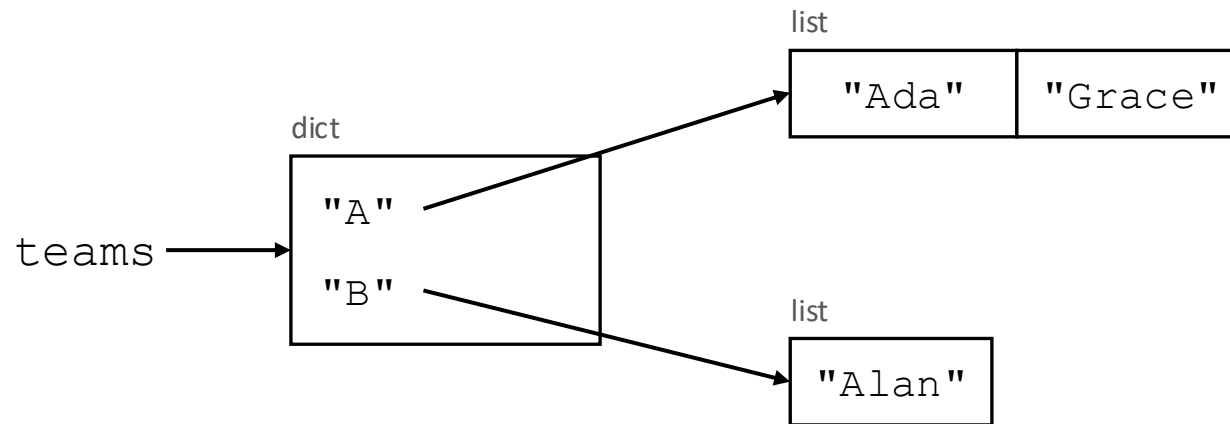
```
print(value_counts("banana"))
```

```
{'b': 1, 'a': 3, 'n': 2}
```

INSIDE A DICTIONARY

- Like a list, a dictionary holds references (arrows), not the values themselves
- Each key has an arrow to its value
- When a value is a list, the arrow goes out to a separate list object

```
teams = {"A": ["Ada", "Grace"],  
        "B": ["Alan"]}
```

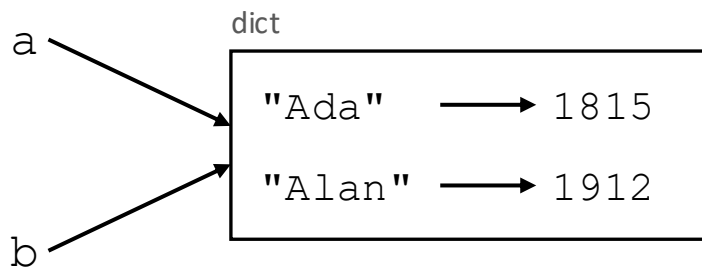


DICTIONARIES ALIAS TOO

- `b = a` doesn't copy the dictionary; it gives it a second name
- Change it through one name and the other name sees it
- Same as lists last week

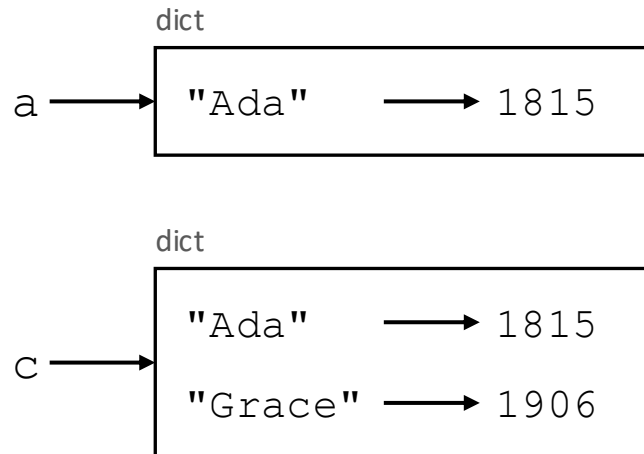
```
a = {"Ada": 1815}
b = a
b["Alan"] = 1912
print(a)
print(a is b)
```

```
{'Ada': 1815, 'Alan': 1912}
True
```



MAKING A COPY

- `dict(a)` or `a.copy()` makes a new dictionary with the same items
- Now a change through one name doesn't affect the other



```
a = {"Ada": 1815}
c = dict(a)
c["Grace"] = 1906
print(a)
print(c)
print(a is c)
```

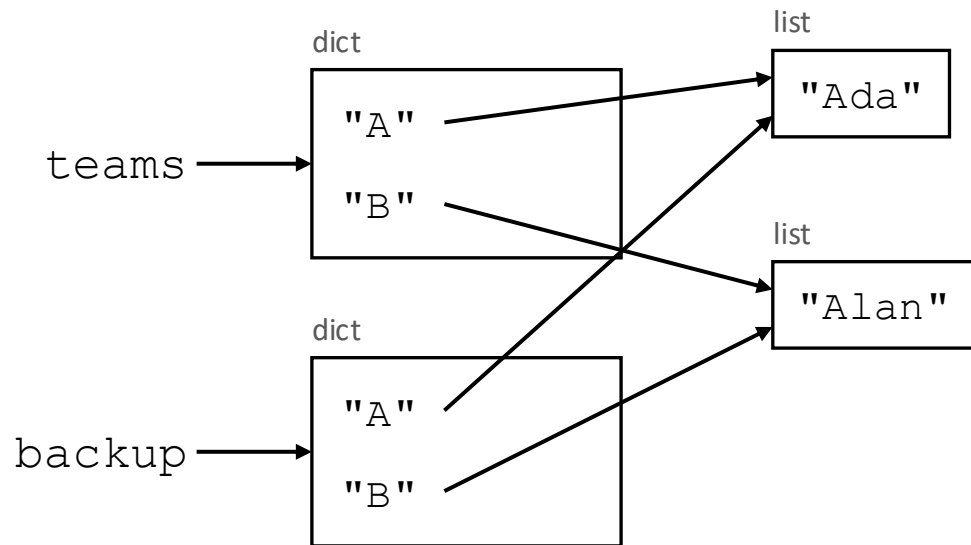
```
{'Ada': 1815}
{'Ada': 1815, 'Grace': 1906}
False
```

COPIES ARE SHALLOW

- `dict(teams)` makes a new dictionary...
- ...but its arrows point at the **same** lists
- The copy is new; the lists inside it are shared

```
teams = {"A": ["Ada"], "B": ["Alan"]}
backup = dict(teams)
print(teams["A"] is backup["A"])
```

True



CHANGING VS. REPLACING

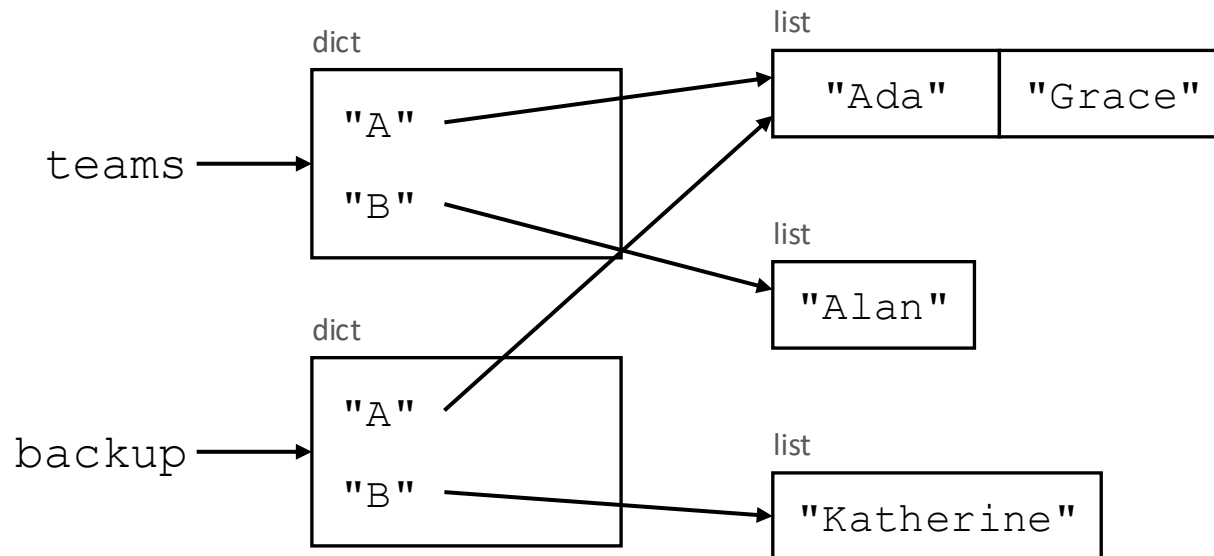
- Changing a shared list shows up through both dictionaries
- Assigning a new value to a key moves only that dictionary's arrow

```
teams = {"A": ["Ada"], "B": ["Alan"]}
backup = dict(teams)
backup["A"].append("Grace")
backup["B"] = ["Katherine"]
print(teams)
print(backup)
```

```
{'A': ['Ada', 'Grace'], 'B': ['Alan']}
```

```
{'A': ['Ada', 'Grace'], 'B': ['Katherine']}
```

Pause and predict: which change shows up in teams?



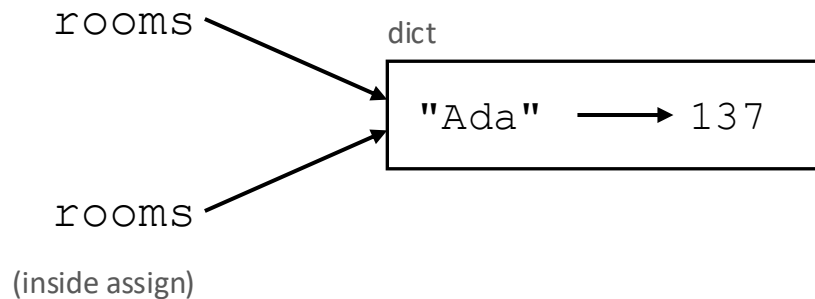
DICTIONARIES AND FUNCTIONS

- The function gets a reference to the caller's dictionary, not a copy
- The parameter is one more alias
- If the function changes the dictionary, the caller sees it

```
def assign(rooms, name, number):  
    rooms[name] = number
```

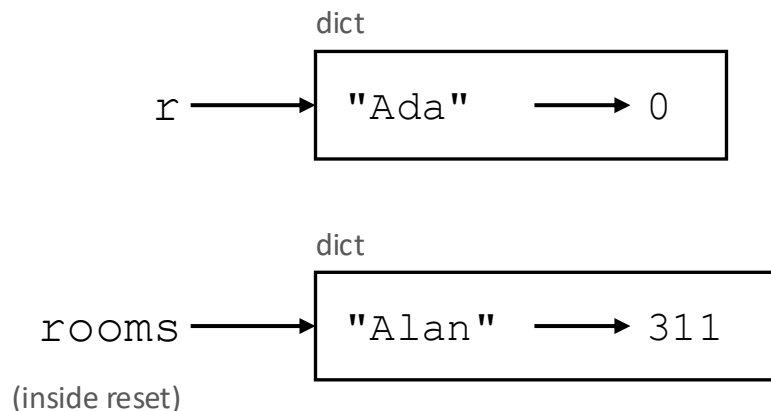
```
rooms = {"Ada": 137}  
assign(rooms, "Grace", 204)  
print(rooms)
```

```
{'Ada': 137, 'Grace': 204}
```



CHANGING VS. REASSIGNING

- Changing the dictionary inside a function: the caller sees it
- Assigning a new dictionary to the parameter: the caller doesn't
- Assignment only moves the local name's arrow



```
def reset(rooms):  
    rooms["Ada"] = 0  
    rooms = {}  
    rooms["Alan"] = 311  
  
r = {"Ada": 137}  
reset(r)  
print(r)
```

`{'Ada': 0}`

Pause and predict: what does `print(r)` show?

REMEMBER CURVE AND CURVED?

- Gradebook: `curve` changed the list and returned nothing; `curved` built a new list
- Functions on dictionaries come in the same two kinds
- **Changes in place:** assigns to a key of the dictionary it was given, returns nothing
- **Builds a new one:** starts from `{}` or `dict(d)`, changes that, returns it

```
def move(rooms, name, number):  
    rooms[name] = number  
  
def moved(rooms, name, number):  
    new_rooms = dict(rooms)  
    new_rooms[name] = number  
    return new_rooms  
  
rooms = {"Ada": 137}  
later = moved(rooms, "Ada", 311)  
print(rooms, later)  
move(rooms, "Ada", 311)  
print(rooms)
```

```
{'Ada': 137} {'Ada': 311}  
{'Ada': 311}
```

A BUG YOU'LL WRITE THIS WEEK

- `move` changes the dictionary and returns `None`
- Saving its result replaces your dictionary with `None`
- If an error mentions `NoneType`, look for an in-place function whose result you saved
- Watch for this with `record` in the scoreboard

```
def move(rooms, name, number):  
    rooms[name] = number  
  
rooms = {"Ada": 137, "Grace": 204}  
rooms = move(rooms, "Ada", 311)  
print(rooms["Grace"])
```

```
TypeError: 'NoneType' object  
is not subscriptable
```

WHAT CAN BE A KEY?

- Last week: "immutable values can be dictionary keys." Here's why that matters
- Strings and numbers can be keys
- A list can be a **value**, but not a **key**

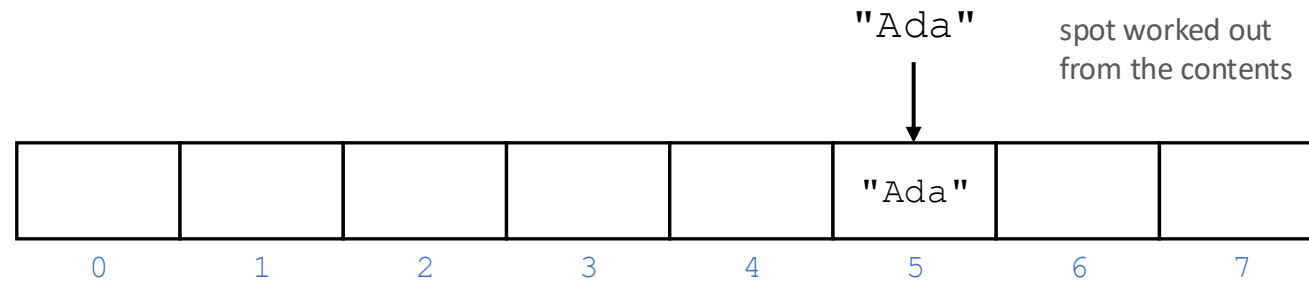
```
d = {}  
d["Ada"] = 1  
d[137] = 2  
d[[1, 1]] = 3
```

```
TypeError: unhashable type: 'list'
```

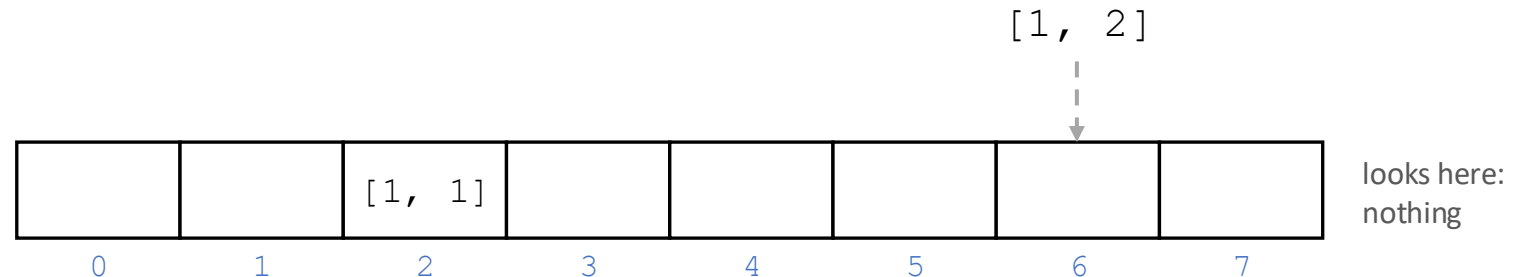
WHY KEYS CAN'T CHANGE

- A dictionary files each key in a spot chosen from the key's contents
- To find the key later, it works out the spot again and looks there
- If the key's contents changed after filing, it would look in the wrong spot
- So keys must be values that can't change

a string key



a list key, filed as [1, 1],
then changed to [1, 2]



WHY DICTIONARIES ARE FAST

- `in` on a list starts at the front and checks every element
- `in` on a dictionary goes straight to the one spot the key could be
- A bigger list means a slower search. A bigger dictionary barely changes

113,783

checks to find out that a word isn't in the word list,
stored as a list

About 1

check to find out the same thing with the words
stored in a dictionary

PREDICT THE OUTPUT

- Work these out on paper before running them
- Then check your answers in Colab
- Draw the arrows!
- Watch the third one closely

```
d = {"a": 1}
d["b"] = d["a"] + 1
d["a"] = 5
print(d)
```

```
rooms = {"Ada": 137}
print(rooms.get("Grace", rooms["Ada"]))
```

```
def add_one(counter):
    counter["n"] = counter.get("n", 0) + 1
    return counter
```

```
x = {}
y = add_one(x)
add_one(y)
print(x)
```

NEXT TIME

- A list can't be a key, because it can change
- In tic-tac-toe, `empty_squares` gives you squares like `[1, 1]`
- So how would you use a square as a key?
- Chapter 11: tuples, which are sequences that can't change

```
moves = {}  
moves[[1, 1]] = "X"      # TypeError  
# what goes here instead?
```